

Basic Quantitative Method: TA Session Week 3

Danny Po-Hsien Kang (康柏賢)

July 17, 2026

Today's agenda

- 1 Remove everything in the environment
- 2 Summary Statistics Table
- 3 Logical expressions
- 4 Data Wrangling

R Programming Challenges

- 7/24, TA Session 8:15-9:00
- Material: Week 1-3 TA Session
- Completed in teams, can use multiple laptops
- Each challenge (40pts): 30 pts for correctness, 10 pts for speed
- Questions based on homework, in-class practices, lecture examples
- Each team has only **one chance** for submitting each question on NTU COOL
- Each member can bring one A4 cheat sheet
- Use of the Internet is completely prohibited during the challenge, except for submitting your work.

Today's Dataset

- Today we will use the following csv files from the OpenIntro Website:
 - `loan_full_schema.csv`: The dataset represents thousands of loans made through the Lending Club platform.
 - `babies.csv`: Pregnancy data between 1960 and 1967 among women in the Kaiser Foundation Health Plan in the San Francisco East Bay area.
 - `earthquake.csv`: Set of notable earthquakes from 1900 to 1999.
- Please import the following datasets into RStudio.

```
loan <- read.csv("data/loan_full_schema.csv")  
babies <- read.csv("data/babies.csv")  
earthquake <- read.csv("data/earthquake.csv")
```

Remove Everything: `rm(list = ls())`

- `rm(list = ls())` removes everything inside the global environment.
 - Useful when you want to re-run the whole code.
- `ls()` returns a character vector of object names in the current environment.

```
x <- 1
y <- "hi"
ls()
# [1] "x" "y"
```

Create Contingency Table: table()

application type	home ownership		
	Rent	Mortgage	Own
individual	3,496	3,839	1,170
joint	362	950	183

Table: Homeownership by application type

Create Contingency Table: table()

- **Syntax:**

```
# dnn = dimension names  
A <- table(..., dnn = ...)
```

- **Example:**

```
loan <- read.csv("loans_full_schema.csv")  
table <- table(loan$application_type,  
               loan$homeownership,  
               dnn = c("application type",  
                       "home ownership"  
                       )  
               )
```

Add Sum Row and Sum Column: addmargins()

application_type	homeownership			
	Rent	Mortgage	Own	Total
individual	3,496	3,839	1,170	8,505
joint	362	950	183	1,495
Total	3,858	4,789	1,353	10,000

Table: Homeownership by application type, with sum row and sum column

Add Sum Row and Sum Column: addmargins()

- **Syntax:**

```
A <- addmargins(table)
```

- **Example:**

```
table_withsum <- addmargins(table)
```

Comparison Operators: >, <, =, !

- > (Greater than): e.g. $5 > 4$ (TRUE)
- >= (Greater or equal): e.g. $3 >= 4$ (FALSE)
- < (Smaller than): e.g. $5 < 4$ (FALSE)
- <= (Smaller or equal): e.g. $3 <= 4$ (TRUE)
- == (Equal to): e.g. $4 == 4$ (TRUE)
- != (Not equal to): e.g. $"H" != "I"$ (TRUE)

Logical Combination: &, &&, |, ||

- AND: & (element wise), && (single element only)

```
x <- c(3, 5, 8)
x > 4 & x < 7 # FALSE TRUE FALSE
x > 4 && x < 7 # Error: 'length = 3' in coercion to 'logical(1)'
```

- OR: | (element wise), || (single element only)

```
x <- c(3, 5, 8)
x > 4 | x < 7 # TRUE TRUE TRUE
x > 4 || x < 7 # Error: 'length = 3' in coercion to 'logical(1)'
```

Check if object inside a vector or list: %in%

- **Syntax:**

```
x %in% y # x and y can be data, vec or list
```

- **Output:**

```
fruit <- c("apple", "banana", "pear")  
basket <- c("banana", "orange", "pear", "kiwi")  
fruit %in% basket # FALSE TRUE TRUE
```

library(dplyr) v.s. library(tidyverse)

- **dplyr** : grammar of data manipulation in R
 - Core functions: `filter()`, `select()`, `mutate()`, `arrange()`, `summarise()`, `group_by()`.
 - Designed for readable, pipeable workflows (`%>%`).
- **tidyverse**: a collection of packages that work together
 - Includes: `dplyr`, `ggplot2`, `tidyr`, `readr`, etc.
 - We used to load `tidyverse` package because usually we need `ggplot2` and `dplyr` packages.
 - Can change it to `library(ggplot2)`, `library(dplyr)` if you want.

Chain Your Operation with Pipeline: %>%

- The pipeline operator %>% is used to chain multiple operations on data together. For example,

```
num <- -3:3  
num %>%  
  sum() %>%  
  print()
```

- Output:**

```
[1] 0
```

Dataset: babies.csv

- case: id number
- bwt: birthweight
- age: mother's age in years
- height (weight): mother's height (weight) in inches

case	bwt	gestation	parity	age	height	weight	smoke
1	120	284	0	27	62	100	0
2	113	282	0	33	64	135	0
3	128	279	0	28	64	115	1
4	123	NA	0	36	69	190	0
5	108	282	0	23	67	125	1

Filter Sample in Dataset: `filter()`

- Suppose we want to filter the mothers with height 62-64. (only first 5 rows are printed)

case	bwt	gestation	parity	age	height	weight	smoke
1	120	284	0	27	62	100	0
2	113	282	0	33	64	135	0
3	128	279	0	28	64	115	1
6	136	286	0	25	62	93	0
7	138	244	0	33	62	178	0

Filter Sample in Dataset: filter()

- **Syntax:**

```
data %>%  
  filter(condition)
```

- **Example:**

```
babies_height_62_64 <- babies %>%  
  filter(height %in% 62:64)
```

- You can use multiple conditions inside filter:

- **Example:**

```
babies_bwt_gt120_smoke <- babies %>%  
  filter(bwt > 120 & smoke == 1)  
babies_bwt_gt120_smoke[1:5, ]
```

Select Variables in Dataset: `select()`

- Suppose we only want to keep columns `bwt` and `age`.

bwt	age
120	27
113	33
128	28
123	36
108	23

Select Variables in Dataset: `select()`

- **Syntax:**

```
data %>%  
  select(cols)
```

- **Example:**

```
babies_age_bwt <- babies %>%  
  select(bwt, age)
```

- You may also select columns by deletion.

- **Example:**

```
babies_wo_case_bwt <- babies %>%  
  select(-c(case, bwt)) # delete column case and bwt
```

Create New Columns by Existing Vars: `mutate()`

- Suppose we want to add a column `ht_plus_wt` that is the sum of `height` and `weight`.

case	bwt	gestation	parity	age	height	weight	smoke	ht_plus_wt
1	120	284	0	27	62	100	0	162
2	113	282	0	33	64	135	0	199
3	128	279	0	28	64	115	1	179
4	123	NA	0	36	69	190	0	259
5	108	282	0	23	67	125	1	192

Create New Columns by Existing Vars: mutate()

- **Syntax:**

```
data %>%  
  mutate(new_var = expr)
```

- **Example:**

```
babies_ht_plus_wt <- babies %>%  
  mutate(ht_plus_wt = height + weight)
```

Create New Columns by Existing Vars: mutate()

- mutate() is very powerful on condition grouping!
 - Extremely useful in practice.
- If the grouping condition is binary, use together with ifelse().
- **Example:**

```
babies_age_gt30 <- babies %>%  
  mutate(age_30 = ifelse(age > 30, "age > 30", "age <= 30"))
```

case	bwt	gestation	parity	age	height	weight	smoke	age_30
1	120	284	0	27	62	100	0	age <= 30
2	113	282	0	33	64	135	0	age > 30
3	128	279	0	28	64	115	1	age <= 30
4	123	NA	0	36	69	190	0	age > 30
5	108	282	0	23	67	125	1	age <= 30

Create New Columns by Existing Vars: mutate()

- If groups ≥ 3 , use together with `case_when()`.
- **Example:**

```
babies_ht_group <- babies %>%
  mutate(ht_group = case_when(
    is.na(height) ~ NA,
    height <= 63 ~ "ht <= 63",
    height > 63 & height < 67 ~ "63 < ht < 67",
    TRUE ~ "ht >= 67"))
```

case	bwt	gestation	parity	age	height	weight	smoke	ht_group
1	120	284	0	27	62	100	0	ht <= 63
2	113	282	0	33	64	135	0	63 < ht < 67
3	128	279	0	28	64	115	1	63 < ht < 67
4	123	NA	0	36	69	190	0	ht >= 67
5	108	282	0	23	67	125	1	ht >= 67

Sort the Datasets: arrange()

- Suppose we want the rows to be in descending order according to the mother's height.

case	bwt	gestation	parity	age	height	weight	smoke
176	122	278	0	31	72	155	1
86	129	274	0	29	71	NA	1
148	160	300	0	29	71	175	1
288	148	279	0	27	71	189	0
294	121	276	1	23	71	152	1

Sort the Datasets: arrange()

- **Syntax:**

```
data %>%  
  arrange(desc(var))
```

- **Example:**

```
babies_ht_desc <- babies %>%  
  arrange(desc(height))
```

- By default, arrange() uses ascending order:

- **Example:**

```
babies_ht_asc <- babies %>%  
  arrange(height)
```

Importance of using %>%

- Use pipeline wisely to make your code more readable and maintainable.
- **Comparison:**

```
# Without %>%  
arrange(select(filter(babies, height >= 60),  
             bwt,  
             height),  
        desc(bwt)  
)
```

```
# With %>%  
babies %>%  
  filter(height >= 60) %>%  
  select(bwt, height) %>%  
  arrange(desc(bwt))
```

Dataset: earthquake.csv

- Data:**

```
> earthquake
```

	year	month	day	richter	area	region	deaths
1	1902	April	19	7.5	...	Guatemala	2000
2	1902	December	16	6.4	Uzbekistan	Russia	4700
3	1903	April	28	7.0	Malazgirt	Turkey	3500
4	1903	May	28	5.8	Gole	Turkey	1000
5	1905	April	4	7.5	Kangra	India	19000

Summarize Data: `summarize()`

- **Syntax:**

```
dataset %>% summarize(  
  mean_var1 = mean(var1),  
  sd_var1 = sd(var1),  
  max_var2 = max(var2),  
  min_var3 = min(var3),  
  ...  
)
```

- **Syntax:**

```
dataset %>% summarize(  
  q75 = c(  
    quantile(var1, 0.75),  
    quantile(var2, 0.75)  
  )  
)
```

Summarize Data: summarize()

- **Example:**

```
earthquake %>%  
  na.omit() %>%  
  summarize(  
    avg_scale = mean(richter),  
    avg_death = mean(deaths)  
  )
```

- **Output:**

```
  avg_scale avg_death  
1  7.166379  18029.1
```

Group Data by Variables: group_by()

- **Syntax:**

```
data %>%  
  group_by(var) %>%  
  summarize(stats = func())
```

- **Example:**

```
earthquake %>%  
  group_by(region) %>%  
  summarize(  
    max_scale = max(richter),  
    min_scale = min(richter),  
    avg_death = mean(deaths)  
  ) %>%  
  arrange(desc(avg_death))
```

Group Data by Variables: group_by()

- **Output:**

	region	max_scale	min_scale	avg_death
	<chr>	<dbl>	<dbl>	<dbl>
1	Peru	7.9	7.3	350700
2	Turkmenistan	7.3	7.3	110000
3	China	8	5.9	40236.
4	Armenia	6.8	6.8	25000
5	Italy	7.2	6.5	21950.
6	Japan	8.4	6.9	20324.
7	Pakistan	8	6.2	13100

Group Data by Variables: `group_by()`

- `group_by()` adds a grouping attribute to the data.
- Many dplyr verbs then behave per group, not on the full data:
 - `summarize()`, `mutate()`, `filter()`, etc.
- Suppose you want to add summary statistics to the dataframe by region for each row:

```
earthquake %>%  
  group_by(region) %>%  
  mutate(  
    max_scale = max(richter),  
    min_scale = min(richter),  
    avg_death = mean(deaths)  
  ) %>%  
  arrange(desc(avg_death)) %>%  
  ungroup()
```


Group Data by Variables: group_by()

• Output:

	year		deaths	region	max_scale	min_scale	avg_death
	<int>	...	<int>	<chr>	<dbl>	<dbl>	<dbl>
1	1946	...	1400	Peru	7.9	7.3	350700
2	1970	...	700000	Peru	7.9	7.3	350700
3	1948	...	110000	Turk...	7.3	7.3	110000
4	1917	...	1800	China	8	5.9	40236.
5	1918	...	1000	China	8	5.9	40236.
6	1920	...	200000	China	8	5.9	40236.
7	1923	...	3500	China	8	5.9	40236.

Group Data by Variables: `group_by()`

- If you do not want the group structure in data, use `ungroup()`.
 - Forgetting using `ungroup()` sometimes can cause tricky problems.
- Suppose you want to find the group with the largest revenue:

```
sales <- data.frame(region = c("N", "N", "S", "S", "S", "S"),
                    store   = c("A", "B", "A", "A", "B", "B"),
                    sales    = c(100, 120, 90, 110, 150, 140))

# forget to ungroup()
by_region_store <- sales %>%
  group_by(region, store) %>%
  summarise(total_sales = sum(sales))

#   region store total_sales
# 1 N      A      100
# 2 N      B      120
# 3 S      A      200
# 4 S      B      290
```

Group Data by Variables: `group_by()`

- Then you use `filter()` to select the rows with largest total sales:

```
by_region_store %>% filter(total_sales == max(total_sales))
```

- Output:

	region	store	total_sales
1	N	B	120
2	S	B	290

- We get two columns here!
- `summarise()` defaults to `.groups = "drop_last"`, which drops the last group.
 - `by_region_store` is now group by region.
- To avoid this, add `ungroup()` after `summarise()`
 - or use `summarise(..., .groups = "drop")`